VetCare

Handover para o back-end

O contrato que o front assume da API (requests e responses), o que mudou em relação ao que já existe e tudo o que precisa ser entregue no VetCare-api.

DATA
2 de novembro de 2026

FRONT EM PRODUÇÃO
vetcare-aal.vercel.app

PARA
VetCare-api

O front (React) está pronto e roda hoje contra um servidor de mock (MSW). Este documento lista **tudo o que o front assume da API** (requests e responses) e **o que precisa ser entregue** no VetCare-api.

1

Deploy (ação necessária)

1. Fazer o deploy do VetCare-api e **passar a URL pública** para quem for configurar o front (eu ou Lucas).
2. Essa URL entra na variável `VITE_API_URL` do projeto no Vercel e `VITE_ENABLE_MSW` passa para `false` (hoje está `true`, então o front de produção usa a API simulada). Depois disso o front é redeployado.
3. **Front em produção:** `https://vetcare-aal.vercel.app`. Use essa URL como `frontEndUrl` nos links de convite (`https://vetcare-aal.vercel.app/convite/{token}`) e no CORS.
4. Variáveis de ambiente do back que precisam existir no deploy: `DATABASE_URL`, `JWT_SECRET` e as do e-mail de convite. Rodar as migrations e o seed (seção 7) no banco de produção.

2

Convenções gerais

Autenticação (JWT)

- O login devolve um JWT em `access_token`. O front guarda esse valor em `localStorage` (chave `vetcare.token`) e **não o decodifica**; os dados do usuário vêm do campo `user` da resposta do login e do aceite do convite, e de `GET /users/me` ao recarregar a página.
- Em **toda** requisição depois do login o front envia:

```
Authorization: Bearer <access_token>
```

- Não há cookies nem refresh token. Rotas públicas (sem header): `POST /auth/login` e `POST /users/invite/{token}/accept`.
- `401` = token ausente, inválido ou expirado (o front faz logout e volta para `/login`). Usuário logado sem permissão para a rota = `403`. Hoje o JWT expira em 1h; ao expirar, o usuário volta ao login (aceitável).
- Usuário `DEACTIVATED` com token ainda válido deve receber `401` em `GET /users/me`.

Headers e CORS

Toda requisição leva `Content-Type: application/json` (inclusive GET e DELETE, sem body). Libere no CORS:

- Origens: `https://vetcare-aal.vercel.app` e `http://localhost:5181` (dev).
- Métodos: `GET, POST, PATCH, DELETE, OPTIONS`.
- Headers: `Authorization, Content-Type`.

Hoje o CORS não está habilitado em `main.ts`.

Erros

Corpo JSON com `message` em português. Pode ser uma string ou, como o `ValidationPipe` do Nest já faz por padrão, um **array de strings**: nesse caso o front mostra só a primeira mensagem. Não é preciso customizar o pipe.

```
{ "message": "Já existe um tutor com este CPF." }
```

STATUS	USO
400	validação
401	sem sessão, token inválido, credenciais inválidas
403	sem permissão para a rota
404	não encontrado (ou soft-deleted)
409	conflito (CPF ou e-mail duplicado)

Paginação e busca

- `page` começa em 1 (padrão 1). `search` é opcional; vazio equivale a ausente.
- A resposta das listas é sempre o envelope abaixo. `total` é o número de registros que casam com o `search` (não o tamanho da página); `pageSize` é o tamanho da página (o front lê o valor, pode ser 8).

```
{ "data": [ ... ], "total": 20, "page": 1, "pageSize": 8 }
```

Formatos

CAMPO	FORMATO
`cpf`, **`phone`**	só dígitos (ex.: `"12345678901"`); o front aplica a máscara
`birthdate`	`"YYYY-MM-DD"`
`weight`	número em kg (ex.: `4.2`)
`sex`	`"M"` ou `"F"`
`species`	`"DOG"` ou `"CAT"` (a UI mostra "Cão" e "Gato")
timestamps	ISO 8601 com timezone (ex.: `"2026-11-02T14:30:00.000Z"`)
`id`	string (UUID). **Todo objeto de usuário devolvido pela API inclui `id`**: login, `GET /users/me`, aceite do convite e a lista de usuários.
`role`	`"ADMIN"`, `"RECEP"` ou `"VET"` (o front só oferece `ADMIN` e `RECEP`)

Regra de senha forte

Aplicada no aceite do convite. O front valida antes de enviar; **a API deve aplicar a mesma regra** (`400` com `message` em português se falhar):

- mínimo de 8 caracteres;
- pelo menos uma letra minúscula, uma maiúscula, um número e um símbolo.

No `POST /auth/login` **não** valide a força da senha (contas antigas ou do seed podem não seguir a regra); só exija o campo preenchido.

3

Autenticação

`POST /auth/login` (público)

Request body:

```
{ "email": "admin@vetcare.com.br", "password": "string" }
```

Response `200`:

```
{
  "access_token": "eyJhbGciOi...",
  "user": {
    "id": "uuid",
    "name": "Renata Freitas",
    "email": "admin@vetcare.com.br",
    "role": "ADMIN"
  }
}
```

- `401` para e-mail inexistente, senha errada, usuário `DEACTIVATED` ou `PENDING_INVITE`. Use a **mesma** mensagem nos casos (ex.: "E-mail e/ou senha incorretos."), para não revelar quais e-mails existem. O front mostra a sua própria mensagem no 401 do login.
- Atualiza `lastAccessAt` do usuário.

`GET /users/me` (autenticado)

Sem body e sem query. Response `200` (sempre com `id`):

```
{
  "id": "uuid",
  "name": "Renata Freitas",
  "email": "admin@vetcare.com.br",
  "role": "ADMIN"
}
```

`401` se o token for inválido/expirado ou o usuário estiver desativado. É chamado a cada recarga da página para reidratar a sessão.

4

Usuários (todas as rotas exigem `ADMIN`, exceto o aceite do convite)

Objeto `User` (nunca inclui senha nem token de convite):

```
{
  "id": "uuid",
  "name": "Juliana Prado",
  "email": "recepcao@vetcare.com.br",
  "role": "RECEP",
  "status": "ACTIVE",
  "lastAccessAt": "2026-11-01T10:00:00.000Z",
  "inviteSentAt": null
}
```

- `status`: `"ACTIVE" | "PENDING_INVITE" | "DEACTIVATED"` (exatamente essa grafia).
- `lastAccessAt` e `inviteSentAt`: ISO ou `null`.

`GET /users?search=&page=`

Query: `search` (string, opcional) casa **nome ou e-mail**, parcial e sem diferenciar maiúsculas; `page` (inteiro). Response `200`: `{ data: User[], total, page, pageSize }`.

`POST /users/invite`

Request body:

```
{ "name": "Camila Duarte", "email": "camila@vetcare.com.br", "role": "RECEP" }
```

Cria o usuário com `status: PENDING_INVITE` (sem senha) e `inviteSentAt = agora`, gera um token de convite e envia e-mail com `https://vetcare-aal.vercel.app/convite/{token}`. Response `201`: o `User` criado. E-mail já cadastrado: `409` `{ "message": "Já existe um usuário com este e-mail." }`.

`POST /users/invite/{token}/accept` (público)

Request body:

```
{ "password": "Senha@Forte1" }
```

Response `200` (mesmo formato do login):

```
{
  "access_token": "eyJhbGciOi...",
  "user": {
    "id": "uuid",
    "name": "Camila Duarte",
    "email": "camila@vetcare.com.br",
    "role": "RECEP"
  }
}
```

O usuário passa a `ACTIVE` e o token de convite é invalidado. O front usa o `user` da resposta para iniciar a sessão, sem chamar `GET /users/me`. Token inexistente, usado ou expirado: `404` (ou 400/410) com `message` em português, ex.: "Convite inválido ou expirado." Senha fraca: `400`.

`POST /users/{id}/deactivate`

Sem body. Response `200` com o `User` atualizado (`status: DEACTIVATED`) ou `204`. O usuário não consegue mais logar (nem usar tokens já emitidos). **Se o usuário tinha um convite pendente, o convite é cancelado**: o link `/convite/{token}` passa a responder como inválido (`404`), e o `accept` não pode reativar o usuário.

`PATCH /users/{id}`

Request body:

```
{ "name": "Camila Duarte", "role": "ADMIN" }
```

Atualiza só `name` e `role` (e-mail é ignorado, não é editável). Response `200`: o `User` atualizado.

5

Tutores (`PetOwner`) e pets (todas as rotas exigem token; `ADMIN` e `RECEP`)

O front usa os nomes da API (`pet-owners` , `petOwner` , `petOwnerId`); o texto da interface continua "tutor".

```
// PetOwner
{
  "id": "uuid", "name": "Maria Silva", "cpf": "12345678901",
  "phone": "11999998888", "email": "maria@email.com", "address": "Rua A, 10",
  "pets": [ /* Pet sem o campo petOwner */ ]
}

// Pet
{
  "id": "uuid", "name": "Mel", "species": "CAT", "breed": "SRD", "sex": "F",
  "birthdate": "2021-07-02", "weight": 4.2, "notes": "Alergia a dipirona",
  "petOwner": { /* PetOwner sem o campo pets */ }
}
```

`notes` são alertas clínicos (alergias, doenças crônicas, cuidados no manejo); pode ser string vazia. Para evitar referência circular, `PetOwner.pets[]` vem sem `petOwner` e `Pet.petOwner` vem sem `pets` .

Tutores

ROTA	QUERY / BODY	RESPONSE
`GET /pet-owners`	Query: `search` , `page` . `search` casa nome, CPF, telefone (também só por dígitos) ou nome de um pet.	`200` `{ data: PetOwner[], total, page, pageSize }`
`POST /pet-owners`	Body abaixo	`201` `PetOwner` (com `pets`)
`GET /pet-owners/{id}`	—	`200` `PetOwner` com `pets` . `404` se não existe ou foi removido.
`PATCH /pet-owners/{id}`	`{ "name", "phone", "email", "address" }` (CPF não é editável; ignorar se vier)	`200` `PetOwner`
`DELETE /pet-owners/{id}`	—	`204` (ou `200`). **Soft delete**, em cascata para os pets do tutor. Some das listas e do `GET` por id.

Body do `POST /pet-owners`:

```
{
  "name": "Maria Silva",
  "cpf": "12345678901",
  "phone": "11999998888",
  "email": "maria@email.com",
  "address": "Rua A, 10",
  "pets": [
    {
      "name": "Mel",
      "species": "CAT",
      "breed": "SRD",
      "sex": "F",
      "birthdate": "2021-07-02",
      "weight": 4.2,
      "notes": ""
    }
  ]
}
```

`pets` é opcional (omitido quando o usuário não cadastra o primeiro pet). Tutor e pets devem ser criados na mesma transação. **CPF duplicado:** `409` `{ "message": "Já existe um tutor com este CPF." }` (o front exibe o texto da API).

Pets

ROTA	QUERY / BODY	RESPONSE
`GET /pets`	Query: `search`, `page`. `search` casa nome do pet, raça ou nome do tutor.	`200` `{ data: Pet[], total, page, pageSize }`, cada pet com `petOwner`
`POST /pets`	`{ "name", "species", "breed", "sex", "birthdate", "weight", "notes", "petOwnerId" }`	`201` `Pet`
`GET /pets/{id}`	—	`200` `Pet`. `404` se não existe ou foi removido.
`PATCH /pets/{id}`	`{ "name", "species", "breed", "sex", "birthdate", "weight", "notes" }` (o dono não é editável)	`200` `Pet`
`DELETE /pets/{id}`	—	`204` (ou `200`). Soft delete.

`POST /pets` com `petOwnerId` inexistente ou removido: `404`.

6

O que o back precisa mudar ou criar

Já existe, mas precisa mudar

1. **Login:** hoje devolve `{ access_token }`. Passar a devolver `{ access_token, user: { id, name, email, role } }`.
2. **Mensagem de 401:** unificar (hoje há duas versões com capitalização diferente) e bloquear login de usuário desativado ou com convite pendente.
3. `User`: adicionar `name`, `status`, `lastAccessAt`, `inviteSentAt` e o token de convite. Nas respostas, expor o campo como `role` (a entidade chama `profile`; o `ResponseUserDto` já usa `role`).
4. `GET /users`: hoje exige `VET` e devolve uma string. Passar a exigir `ADMIN`, com busca e paginação. O `POST /users` atual (cria usuário com senha) não faz parte do contrato; pode ficar só para o seed/uso interno, ou ser removido.
5. **Schema de tutor/pet:** renomear para `PetOwner`/`Pet` e ajustar:
   - `adress` → `address`;
   - `phone String[]` → `String`;
   - `birth` → `birthdate`;
   - adicionar `Pet.name`, `weight`, `notes` e `deletedAt` (nos dois);
   - **remover o `@unique` de `Animal.tutorId`**: hoje um tutor só pode ter um pet;
   - enum `sex`: `M` / `F`.
6. **CORS** em `main.ts` (seção 2).
7. **Controllers `/tutors` e `/vets`:** são scaffolding (retornam texto) e os DTOs estão vazios (com `whitelist` + `forbidNonWhitelisted`, qualquer body seria rejeitado). Trocar por `/pet-owners` e `/pets`.
8. **IDs:** nas rotas de users e tutors o código faz `+id`; os IDs são UUID, então vira `NaN`. Usar o `id` como string.

Não existe ainda

- `GET /users/me`.
- Fluxo de convite: `POST /users/invite`, `POST /users/invite/{token}/accept` e o envio de e-mail.
- `POST /users/{id}/deactivate` e `PATCH /users/{id}`.
- Busca, paginação e o envelope `{ data, total, page, pageSize }`.
- Soft delete com cascata.
- Tradução do erro de CPF duplicado (hoje só existe a constraint `unique` no banco) para `409`.
- Validação de senha forte no aceite do convite.

7

Seed (obrigatório, inclusive em produção)

PERFIL	NOME	E-MAIL	SENHA
`ADMIN`	Renata Freitas	`admin@vetcare.com.br`	`password`
`RECEP`	Juliana Prado	`recepcao@vetcare.com.br`	`password`

Ambos `ACTIVE`. A senha `password` não passa na regra de senha forte; por isso o login não valida força. Troque por senhas fortes antes de uso real.

8

Como testar com o front

Em `vetcare-front`, copie `.env.example` para `.env` e aponte `VITE_API_URL` para a API. O MSW (API simulada no navegador) só liga com `VITE_ENABLE_MSW=true`; em `npm run dev` ele vem ligado por `.env.development`, então para usar a API real crie um `.env.local` com `VITE_ENABLE_MSW=false`. No Vercel, por enquanto `VITE_ENABLE_MSW=true` (o front de produção ainda usa a API simulada); quando a API real estiver no ar, troque para `false` junto com o `VITE_API_URL`.